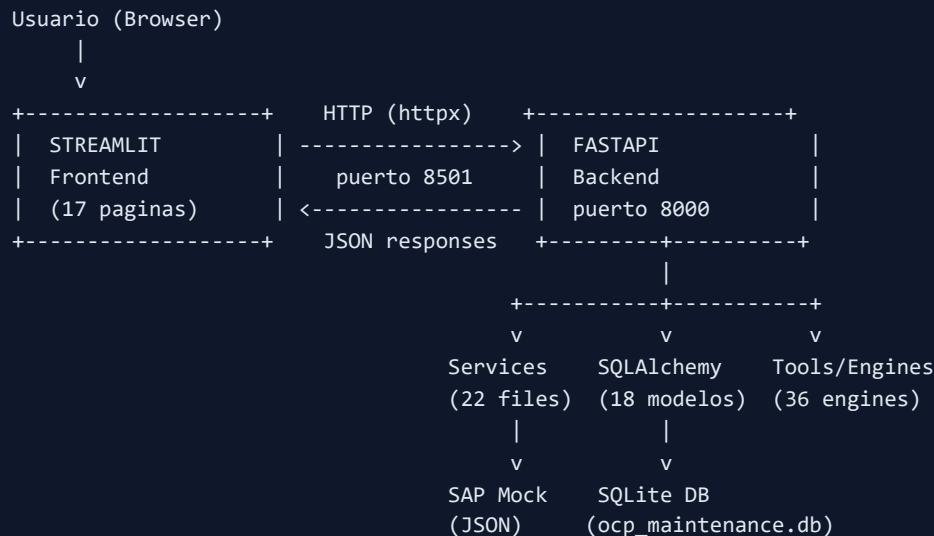


Arquitectura del Proyecto — OCP Maintenance AI

Vista General



1. Frontend — `streamlit_app/`

Entry point: `streamlit_app/app.py` — carga el sidebar, i18n, y el CSS base.

17 paginas en `streamlit_app/pages/`, organizadas por modulo:

Modulo	Paginas	Funcion
Captura	01 Field Capture, 02 Work Requests	Tecnico en campo captura datos (texto/voz/foto) -> se genera Work Request
Planificacion	03 Backlog, 04 Scheduling, 05 Planner	Backlog priorizado, Gantt, nivelacion de recursos
Ejecutivo	06 Dashboard, 13 Overview	KPIs, health score, metricas resumen
R8 Strategy	07 Hierarchy, 08 Criticality, 09 FMEA, 10 Strategy	Descomposicion 6 niveles -> Criticidad GFSN -> FMEA 72 combos -> Asignacion de estrategia
Reliability	11 Analytics, 14 Reliability, 16 FMECA, 17 Defect Elimination	Weibull, Pareto, RCA, CAPA
SAP/Reports	12 SAP Review, 15 Reports & Data	Export/import SAP, reportes Excel/PDF

Componentes reutilizables — `streamlit_app/components/`

Archivo	Funcion
<code>charts.py</code>	Plotly (Weibull, Pareto, tendencias)
<code>tables.py</code>	Tablas interactivas con filtros
<code>forms.py</code>	Formularios (criticidad, tareas)
<code>tree_panel.py</code>	Arbol jerarquico de equipos
<code>feedback.py</code>	Rating para contenido generado por IA

API Client — `streamlit_app/api_client.py`

Todas las llamadas HTTP al backend. Optimizado con:

- **Cache:** `@st.cache_data(ttl=60)` en todos los GET
- **Connection pooling:** 10 keep-alive, 20 max connections
- **Timeout:** 10s
- **Invalidacion automatica:** toda mutacion (POST/PUT/DELETE) limpia el cache

i18n — `streamlit_app/i18n/`

Soporte multilenguaje: ingles (`en.json`), frances (`fr.json`), espanol (`es.json`), arabe (`ar.json` con RTL).

CSS — `streamlit_app/style.py`

Sistema de estilos centralizado:

- `apply_style()` — CSS base OCP (usado por TODAS las paginas)
 - `apply_capture_style()` — CSS especifico para Field Capture
 - `apply_work_request_style()` — CSS especifico para Work Requests
 - Font Inter via `<link>` no-bloqueante
-

2. Backend — `api/`

Entry point: `api/main.py` — FastAPI app con CORS, health check, 18 routers.

Arquitectura en 3 capas

```

Routers (18)  ->  Services (22)  ->  Database (SQLAlchemy)
|              |              |
Endpoints     Logica          18 modelos ORM
REST         de negocio       SQLite/PostgreSQL
```

Routers — [api/routers/](#)

Router	Modulo	Proposito
hierarchy.py	Modulo 0	CRUD jerarquia equipos, 6 niveles
criticality.py	Modulo 0	Evaluacion criticidad GFSN
fmea.py	Modulo 0	Creacion FMEA, 72 combinaciones
tasks.py	Modulo 0	CRUD tareas de mantenimiento
capture.py	Modulo 1	Captura de campo, preview OCR
work_requests.py	Modulo 1	Creacion/validacion Work Requests
planner.py	Modulo 2	Nivelacion recursos, Gantt
backlog.py	Modulo 2	Agrupacion backlog por equipo/prioridad
scheduling.py	Modulo 3	Scheduling con restricciones
work_packages.py	Modulo 1	Ensamblaje paquetes de trabajo
sap.py	Integracion	Data mock SAP, export/import
reliability.py	Modulo 4	Weibull, prediccion de fallas
rca.py	Modulo 4	Root cause analysis, 5-Why
reporting.py	Modulo 5	Generacion reportes (Excel, PDF)
analytics.py	Modulo 5	Dashboards real-time, health scores
dashboard.py	Modulo 5	Metricas ejecutivas
admin.py	Admin	Seed DB, health check, status

Base de datos — [api/database/](#)

connection.py: SQLAlchemy 2.0+ con SQLite (dev) o PostgreSQL (prod). Connection pooling, foreign keys enforced.

models.py — 18 modelos ORM:

Modelo	Proposito
<code>PlantModel</code>	Sitio/planta
<code>HierarchyNodeModel</code>	6 niveles (Plant -> Area -> System -> Equipment -> Sub-Assembly -> Maintainable Item)
<code>CriticalityAssessmentModel</code>	Matriz GFSN (Gravity x Frequency x Severity x Non-detect)
<code>FunctionModel</code>	Funciones primarias/secundarias/protectivas
<code>FunctionalFailureModel</code>	Modos de perdida de funcion
<code>FailureModeModel</code>	72 combinaciones FMEA
<code>MaintenanceTaskModel</code>	Tareas (condition/time/run-based)
<code>WorkPackageModel</code>	Paquetes de trabajo
+ Materials, WorkOrders...	

Configuracion — `api/config.py`

Variables de entorno (`.env`):

- `DATABASE_URL` — SQLite o PostgreSQL
- `SAP MOCK DIR` — Directorio data SAP mock
- `ANTHROPIC_API_KEY` — Para agentes IA (opcional)
- `CORS_ORIGINS` — Origenes permitidos
- `LOG_LEVEL` , `DEBUG`

3. Tools/Engines — `tools/`

La logica pesada de calculo, separada del API.

Engine	Que hace
criticality_engine.py	Calculo GFSN
rcm_decision_engine.py	Arbol de decision RCM (Evidente? Prevenible? Seguro?)
hierarchy_builder_engine.py	Construye jerarquia 6 niveles desde libreria de equipos
weibull_engine.py	Estimacion Weibull, MTBF/MTTF
scheduling_engine.py	Optimizacion con restricciones de recursos
fmeca_engine.py	Analisis criticidad-efecto
rca_engine.py	5-Why analysis
sap_export_engine.py	Formato SAP PM (Maintenance Item, Task List)
health_score_engine.py	Indice de salud de equipo (0-100)
kpi_engine.py	Agregacion KPIs (MTTR, MTBF, disponibilidad)
planning_kpi_engine.py	Metricas de schedule (SPI, CPI)
work_package_assembly_engine.py	Agrupacion y secuenciacion de tareas
spare_parts_engine.py	Expansion BOM, requerimiento inventario
backlog_grouper.py	Clustering por prioridad
pareto_engine.py	Analisis 80/20 frecuencia de fallas
variance_detector.py	Desviacion actual vs. planificado
capa_engine.py	Acciones correctivas/preventivas
data_export_engine.py	Export multi-formato (Excel, JSON, CSV)
data_import_engine.py	Import con validacion de templates
notification_engine.py	Alertas y escalaciones
+ 16 mas	OCR, Material Mapper, LCC, RBI, MOC, Jackknife, Shutdown...

Processors (5) — `tools/processors/`

Archivo	Funcion
<code>field_capture_processor.py</code>	Parseo/validacion de capturas OCR
<code>backlog_optimizer.py</code>	Ordenamiento heuristico de backlog
<code>planner_engine.py</code>	Scheduling basado en restricciones
<code>gantt_generator.py</code>	Generacion de graficos Gantt
<code>pii_redactor.py</code>	Enmascaramiento de datos sensibles

Validators (4) — `tools/validators/`

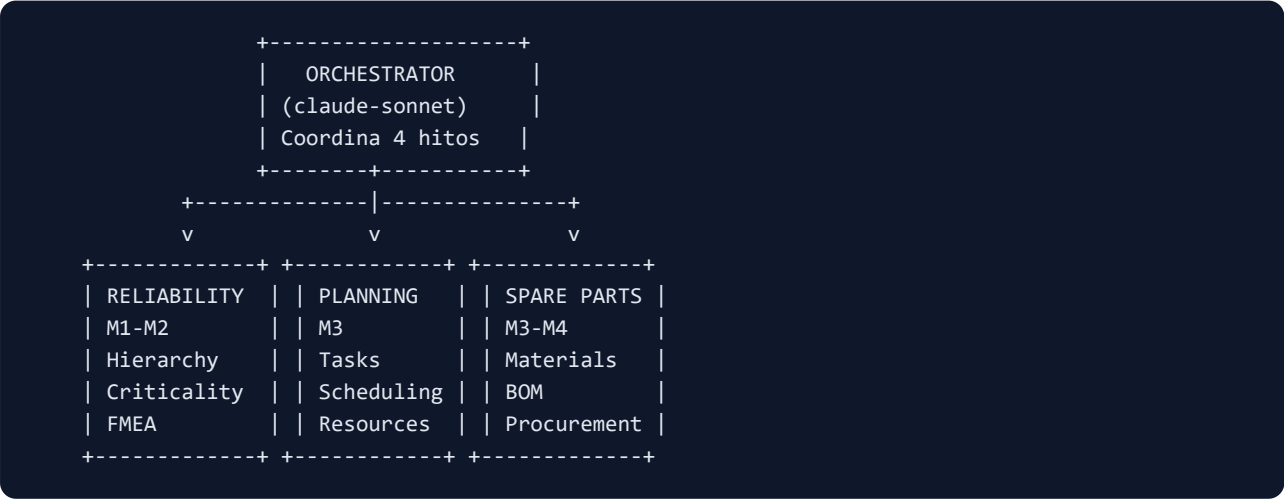
Archivo	Funcion
<code>quality_validator.py</code>	Reglas de negocio
<code>naming_validator.py</code>	Convenciones ISO14224
<code>confidence_validator.py</code>	Umbrales de confianza IA (>0.85)

Schemas — `tools/models/schemas.py`

Modelos Pydantic v2 — source of truth para todas las estructuras de datos del proyecto.

4. AI Agents — `agents/`

4 agentes especializados (requieren `ANTHROPIC_API_KEY`)



Workflow de 4 Milestones

Cada milestone tiene un gate de aprobacion humana:

Milestone	Contenido	Estado
M1	Hierarchy Decomposition + Criticality Assessment	PENDING -> IN_PROGRESS -> PRESENTED -> APPROVED
M2	FMEA (72 combos) + RCM Decisions	Mismo flujo
M3	Strategy + Tasks + Work Packages	Mismo flujo
M4	SAP Upload Package	Mismo flujo

Estructura interna

Directorio	Proposito
agents/definitions/	Configuracion de cada agente (modelo, prompt, tools)
agents/orchestration/	Workflow engine, session state, milestones, checkpoints
agents/tool_wrappers/	30 wrappers que conectan tools/engines como herramientas MCP
agents/_shared/	Utilidades compartidas (loader, prompts)

5. SAP Mock — sap_mock/

Data JSON simulando SAP PM (Plant Maintenance):

Archivo	Contenido
equipment_master.json	~25 equipos (EQUNR, descripcion, fabricante)
functional_locations.json	~30 ubicaciones funcionales (TPLNR)
maintenance_plans.json	~20 planes con ciclos
materials_bom.json	~100+ entradas BOM con costos
work_orders.json	~50+ ordenes historicas (AUFNR, fecha, costo)

6. Templates — templates/

14 templates Excel para import/export:

#	Archivo	Proposito
01	equipment_hierarchy.xlsx	Definicion jerarquia 6 niveles
02	criticality_assessment.xlsx	Matriz scoring GFSN
03	failure_modes.xlsx	Matriz FMEA 72 combos
04	maintenance_tasks.xlsx	Master de tareas con estrategia
05	work_packages.xlsx	Agrupacion paquetes de trabajo
06	work_order_history.xlsx	Data historica de mantenimiento
07	spare_parts_inventory.xlsx	Inventario actual
08	shutdown_calendar.xlsx	Ventanas de exclusion
09	workforce.xlsx	Disponibilidad de cuadrillas
10	field_capture.xlsx	Template captura de campo
11	rca_events.xlsx	Registros root cause analysis
12	planning_kpi_input.xlsx	Inputs KPI de schedule
13	de_kpi_input.xlsx	Inputs metricas defect elimination
14	maintenance_strategy.xlsx	Output completo de estrategia (77+ columnas)

7. Skills — skills/

Definiciones de habilidades para los agentes IA:

Carpeta	Proposito
<code>00-knowledge-base/</code>	Docs de arquitectura, guias, metodologia
<code>02-maintenance-strategy-development/</code>	RCM, FMEA, criticidad
<code>02-work-planning/</code>	Nivelacion recursos, scheduling
<code>03-reliability-engineering-and-defect-elimination/</code>	Weibull, RCA, CAPA
<code>04-cost-analysis/</code>	LCC, presupuesto
<code>05-general-functionalities/</code>	Import/export, validacion, notificaciones
<code>06-orchestation/</code>	Coordinacion workflow, KPIs

8. Tests — `tests/`

1,577 tests en 110 archivos:

Categoria	Archivos	Que cubre
Unit tests	77	Cada engine, service, schema, validator
API tests	16 (<code>test_api/</code>)	Endpoints CRUD, validacion, errores
Integration	2 (<code>test_integration/</code>)	Workflow end-to-end
Security	6	XSS, injection, serialization, data exposure
E2E	6 (<code>e2e/</code>)	Playwright — interaccion con paginas Streamlit

Configuracion en `pytest.ini` : `--ignore=tests/e2e` (E2E requiere Playwright aparte).

9. Deployment — Docker

`docker-compose.yml` — 3 servicios

```
+-----+ +-----+ +-----+
| ocp-nginx | | ocp-frontend | | ocp-backend | |
| (nginx:alpine) |---->| (streamlit) | | (gunicorn + |
| Puerto: 8080 | | Puerto: 8501 |---->| uvicorn) |
| | | | | | Puerto: 8000 |
+-----+ +-----+ +-----+
Reverse proxy Frontend API + DB
```

Servicio	Comando	Puerto
ocp-backend	<code>gunicorn api.main:app -w 4 -k uvicorn.workers.UvicornWorker</code>	8000
ocp-frontend	<code>streamlit run streamlit_app/app.py --server.port 8501</code>	8501
ocp-nginx	nginx reverse proxy	8080

Volumenes: `ocp_db_data` (SQLite), `ocp_sap_data` (SAP mock)

Quick start: `bash start.sh` -> `docker compose up --build -d`

10. Flujo de datos completo (ejemplo)

```
Tecnico en campo captura falla
|
v
[Page 01: Field Capture] (texto/voz/foto)
|
v  api_client.create_capture()
|  POST /api/v1/capture/
v
[capture_service.py] -> DB
|
v
Se crea Work Request
|
v
[Page 02: Work Requests] -> Validacion/Aprobacion
|
v
Entra al Backlog
|
v
[Page 03: Backlog] -> Priorizado
|
v
[Page 05: Planner] -> Programa recursos
|
v
[Page 04: Scheduling] -> Genera Gantt
|
v
Ejecutado en campo
|
v
[Page 14: Reliability] -> Analisis Weibull de tendencia
|
v
[Page 11: Analytics] -> Actualiza KPIs
|
v
[Page 06: Dashboard] -> Resumen ejecutivo
```

11. Estructura de carpetas

```
ASSET-MANAGEMENT-SOFTWARE-master/
|
|-- .dockerignore
|-- .env.example
|-- .gitignore
|-- Dockerfile
|-- docker-compose.yml
|-- nginx.conf
|-- pytest.ini
|-- requirements.txt
|-- start.sh
|
|-- api/                                # Backend FastAPI
|   |-- main.py                        # Entry point
|   |-- config.py                      # Settings (.env)
|   |-- seed.py                        # Database seeding
|   |-- database/
|       |-- connection.py             # SQLAlchemy engine
|       +-- models.py                 # 18 ORM models
|   |-- routers/                       # 18 REST routers
|   +-- services/                     # 22 business logic services
|
|-- streamlit_app/                     # Frontend Streamlit
|   |-- app.py                         # Entry point
|   |-- api_client.py                 # HTTP client (cached)
|   |-- style.py                      # CSS system
|   |-- components/                  # 5 reusable UI components
|   |-- i18n/                         # Translations (en, fr, es, ar)
|   +-- pages/                       # 17 pages (01-17)
|
|-- agents/                           # AI Multi-Agent System
|   |-- run.py                        # Agent runner
|   |-- definitions/                  # 4 agent configs + prompts
|   |-- orchestration/                # Workflow, milestones, checkpoints
|   |-- tool_wrappers/                # 30 MCP tool wrappers
|   +-- _shared/                      # Shared utilities
|
|-- tools/                             # Business Logic Layer
|   |-- engines/                      # 36 calculation engines
|   |-- processors/                   # 5 data processors
|   |-- validators/                   # 4 validators
|   |-- generators/                   # Synthetic data
|   +-- models/                       # Pydantic schemas
|
|-- sap_mock/                          # SAP PM mock data (JSON)
|-- skills/                            # Agent skill definitions
|-- templates/                         # 14 Excel templates
|-- scripts/                           # Utility scripts
|-- tests/                             # 1,577 tests (110 files)
|   |-- test_api/                     # API endpoint tests
|   |-- test_integration/              # E2E workflow tests
|   |-- test_processors/              # Processor tests
|   +-- e2e/                          # Playwright browser tests
+-- docs/                             # Documentation
    |-- ARCHITECTURE.md                # Este archivo
```

```
|-- context/      # Documentos de contexto
+-- how-to-guides/ # Guías de usuario
```

12. Dependencias principales

Paquete	Version	Uso
FastAPI	<code>>=0.115</code>	Backend REST API
Uvicorn	<code>>=0.30</code>	ASGI server
Gunicorn	<code>>=22.0</code>	Production server
SQLAlchemy	<code>>=2.0</code>	ORM
Pydantic	<code>>=2.0</code>	Schemas/validacion
Streamlit	<code>>=1.38</code>	Frontend web
Plotly	<code>>=5.22</code>	Graficos interactivos
httpx	<code>>=0.27</code>	HTTP client (con pooling)
Anthropic	<code>>=0.39</code>	Claude API (agentes)
Openpyxl	<code>>=3.1</code>	Excel templates
Pytest	<code>>=8.0</code>	Testing